PricebookCalc Developer Guide

This guide documents every feature in PricebookCalc, explaining how each one works, where the code lives, and how the pieces connect.

Table of Contents

1. Architecture Overview
2. Database Schema
3. System Calculator
4. Repair Calculator
5. Reverse Calculator
6. Rebates (OEM & Utility)
7. ResultsCard Component
8. Calculator Context (Cross-Calculator State Sharing)
9. Settings & Per-Device Cache
10. Quote Saving & History
11. ServiceTitan Integration
12. Push to ServiceTitan (Update Pricebook)
13. Push Queue & Rate Limiting
14. New System Task & New Repair Task
15. PSI Mode Toggle
16. Location-Based Sales Tax
17. Commission & Labor Modes
18. Authentication & Access Control
19. Version Check Banner
20. Navigation & Routing

---

1. Architecture Overview

Tech Stack

Layer	Technology
Frontend	React 18, TypeScript, Vite
Routing	Wouter (client-side)
State	TanStack React Query (server), React Context (cross-calculator)
UI	shadcn/ui (Radix primitives), Tailwind CSS
Charts	Recharts
Backend	Node.js, Express 5, TypeScript (ESM)
Database	PostgreSQL (Neon-backed via Replit)
ORM	Drizzle ORM with drizzle-zod validation

Project Structure

```
client/src/
├── App.tsx                    # Root component, routing, providers
├── contexts/
│   ├── CalculatorContext.tsx  # Shared calculator state (System + Repair)
│   ├── ServiceTitanAuthContext.tsx  # ServiceTitan password auth state
│   └── AdminAuthContext.tsx   # Admin PIN auth state
├── components/
│   ├── Navigation.tsx         # Sidebar navigation with admin panel
│   ├── ResultsCard.tsx      # Shared results display (price, GM%, chart)
│   ├── PasswordGate.tsx     # Password prompt wrapper for protected pages
│   ├── VersionCheck.tsx       # New-version-available banner
│   └── InputMoney.tsx         # Reusable currency input component
├── pages/
│   ├── Calculator.tsx         # System Calculator page
│   ├── RepairCalculator.tsx   # Repair Calculator page
│   ├── ReverseCalculator.tsx  # Reverse Calculator page (System & Repair modes)
│   ├── PushToServiceTitan.tsx # Push page with 3 tabs
│   ├── PushHistory.tsx        # Push history log
│   ├── Tenants.tsx            # Tenant management (CRUD)
│   ├── Locations.tsx          # Location/sales-tax management (CRUD)
│   ├── Settings.tsx           # Global calculator settings
│   └── History.tsx            # Saved quotes list
├── hooks/
│   └── use-settings.ts      # Settings fetch hook with localStorage cache
└── lib/
    └── queryClient.ts         # TanStack Query client setup

server/
├── index.ts                   # Server entry point
├── routes.ts                  # All API route handlers (~1,300 lines)
├── storage.ts                 # Database CRUD operations (Drizzle)
├── db.ts                      # Database connection
├── servicetitan.ts            # ServiceTitan API client class
├── pushProcessor.ts           # Background push processing logic
├── pushQueue.ts               # Push queue with concurrency control
├── pushProgress.ts            # Server-Sent Events for push progress
├── vite.ts                    # Vite dev server integration
└── static.ts                  # Static file serving (production)

shared/
├── schema.ts              # Drizzle table definitions, Zod schemas, types
└── routes.ts                  # Shared API route definitions with Zod
```

Key Design Patterns

- **Shared Route Definitions**: API routes defined once in `shared/routes.ts` with Zod schemas — used by both client and server for type-safe API calls.
- **Settings Singleton**: The `settings` table has exactly one row. Auto-created with defaults if missing.
- **Quote Snapshots**: Saved quotes capture calculated results at the time of saving, preserving history even if global settings change later.
- **CalculatorContext**: React Context that holds both System Calculator state and Repair Calculator state, enabling cross-calculator data sharing (e.g., "Copy from" in Reverse Calculator).

---

2. Database Schema

File: `shared/schema.ts`

Tables

`settings` (Singleton) Stores all global calculator configuration. One row (id=1), auto-created with defaults.

Column	Type	Default	Purpose
`id`	serial	auto	Primary key
`targetMePercent`	numeric	"23.00"	Default M&E target percentage
`salesTaxPercent`	numeric	"6.35"	Default sales tax rate
`laborMode`	text (enum: "hours"/"percent")	"percent"	Which labor input mode to use
`laborHourlyRate`	numeric	"0.00"	Hourly rate when labor mode is "hours"
`laborPercent`	numeric	"7.00"	Labor % when mode is "percent"
`laborBurdenPercent`	numeric	"20.00"	Employer burden applied to labor and commission
`commissionPercent`	numeric	"10.00"	Commission as % of selling price
`commissionBurdenPercent`	numeric	"20.00"	Burden applied to commission
`financingFeePercent`	numeric	"6.00"	Financing fee as % of price
`grossMarginMinPercent`	numeric	"52.00"	Minimum GM% threshold for health indicators

Column	Type	Default	Purpose
serviceTitanPassword	text	“ZephyrHome160”	Password for ServiceTitan section access
adminPin	text	“7437”	PIN for Admin section access

quotes Saved calculation snapshots from the System Calculator.

Column	Type	Default	Purpose
id	serial	auto	Primary key
title	text	“Untitled Quote”	User-given quote name
materialsCost	numeric	required	Input: materials cost
equipmentCost	numeric	required	Input: equipment cost
laborHoursInput	numeric	“10”	Input: labor hours (hours mode)
calculatedPrice	numeric	required	Snapshot: final calculated price
calculatedGrossMargin	numeric	required	Snapshot: GM% at save time
calculatedEstimatedLabor	numeric	required	Snapshot: estimated labor at save time
calculatedMePercent	numeric	required	Snapshot: M&E% at save time
subcontractorCost	numeric	“0.00”	Input: subcontractor cost
craneCost	numeric	“0.00”	Input: crane cost
permitCost	numeric	“0.00”	Input: permit cost
createdAt	timestamp	now()	When quote was saved

tenants ServiceTitan API credentials for multi-tenant support.

Column	Type	Default	Purpose
id	serial	auto	Primary key
name	text	required	Display name
tenantId	text	required	ServiceTitan tenant ID
clientId	text	required	OAuth client ID
clientSecret	text	required	OAuth client secret
appKey	text	required	ServiceTitan app key
locationId	integer	null	FK to locations table (for sales tax lookup)

Column	Type	Default	Purpose
isActive	boolean	true	Whether tenant is enabled
sortOrder	integer	0	Display ordering
createdAt	timestamp	now()	Creation timestamp
updatedAt	timestamp	now()	Last update timestamp

locations Reference library mapping location names to sales tax percentages.

Column	Type	Default	Purpose
id	serial	auto	Primary key
name	text	required	Location display name (e.g., “Hartford CT”)
salesTaxPercent	numeric	“0.00”	Sales tax rate for this location
createdAt	timestamp	now()	Creation timestamp

pushHistory Audit log for ServiceTitan pricebook push operations. Stores a snapshot of all settings used.

Column	Type	Default	Purpose
id	serial	auto	Primary key
tenantId	integer	required	Which tenant was pushed to
status	text (enum)	“pending”	pending / in_progress / completed / failed / cancelled / reverted
categoryIds	text (JSON)	required	JSON array of category IDs that were pushed
targetMePercent	numeric	required	M&E target used for this push
salesTaxPercent	numeric	required	Sales tax rate used
laborMode	text	required	“hours” or “percent”
laborHourlyRate	numeric	required	Hourly rate used
laborHours	numeric	“8”	Default hours per service
laborPercent	numeric	required	Labor % used
laborBurdenPercent	numeric	required	Labor burden % used
commissionPercent	numeric	required	Commission % used
commissionBurdenPercent	numeric	required	Commission burden % used

Column	Type	Default	Purpose
financingFeePercent	numeric	required	Financing fee % used
roundUp	boolean	false	Whether prices were rounded up
processedCount	integer	0	How many services were examined
updatedCount	integer	0	How many services were actually updated
skippedCount	integer	0	How many were skipped (no M&E)
errorCount	integer	0	How many failed
errorMessages	text (JSON)	null	JSON array of error message strings
originalValues	text (JSON)	null	JSON array of {serviceId, originalPrice, originalCost} for revert
createdAt	timestamp	now()	When push was initiated
startedAt	timestamp	null	When processing began
completedAt	timestamp	null	When processing finished

---

3. System Calculator

File: client/src/pages/Calculator.tsx **Route**: /

The System Calculator computes a total selling price for an HVAC system installation based on materials, equipment, and a target M&E percentage.

Inputs

Input	Description
Target M&E %	Dropdown (10%–50% in 0.5% steps). What percentage of the final price should M&E represent.
Materials Cost	Dollar amount of materials
Equipment Cost	Dollar amount of equipment
Labor Hours or Labor %	Depends on labor mode in Settings
Subcontractor	Additional cost added to price
Crane	Additional cost added to price
Permit	Additional cost added to price
Discount	Dollar amount or % of total to subtract

Input	Description
OEM Rebate	Dollar amount added to revenue for GM% (not to price)
Utility Rebate	Dollar amount added to revenue for GM% (not to price)

Calculation Logic (in the `useEffect`, lines 89–175)

```
Step 1: Tax & M&E
  salesTaxRate = parseFloat(settings.salesTaxPercent) / 100
  tax = (materials + equipment) × salesTaxRate
  totalMe = materials + equipment + tax

Step 2: Base Price from M&E Target
  targetMeRate = parseFloat(targetMe) / 100
  basePrice = targetMeRate > 0 ? totalMe / targetMeRate : 0

Step 3: Add Other COGS, Apply Discount
  totalOtherCogs = subcontractor + crane + permit
  priceBeforeDiscount = basePrice + totalOtherCogs

  Discount Mode handling:
    If discountMode is "dollar":
      discountVal = discountInput (raw dollar amount)
    If discountMode is "percent":
      discountVal = priceBeforeDiscount × (discountInput / 100)

  price = max(0, priceBeforeDiscount - discountVal)

Step 4: Commission (always % of job, with burden)
  commissionRate = parseFloat(settings.commissionPercent) / 100
  commissionBurdenRate = parseFloat(settings.commissionBurdenPercent) / 100
  commissionCost = (price × commissionRate) × (1 + commissionBurdenRate)

Step 5: Financing
  financingRate = parseFloat(settings.financingFeePercent) / 100
  financingCost = price × financingRate

Step 6: Labor Cost (depends on settings.laborMode)
  laborBurdenRate = parseFloat(settings.laborBurdenPercent) / 100
  If laborMode is "hours":
    hourlyRate = parseFloat(settings.laborHourlyRate)
    laborCost = (hours × hourlyRate) × (1 + laborBurdenRate)
  If laborMode is "percent":
   laborPercentRate = parseFloat(laborPercent) / 100  (from user input, not settings)
    laborCost = (price × laborPercentRate) × (1 + laborBurdenRate)
```

```
Step 7: Estimated Labor Field (for ServiceTitan import)
  estimatedLabor = laborCost + commissionCost + tax + financingCost

Step 8: Display Price & Gross Margin
  displayPrice = roundUp ? Math.ceil(price) : price
 totalAllCogs = totalMe + commissionCost + financingCost + laborCost + totalOtherCogs
  effectiveRevenue = displayPrice + oemRebate + utilityRebate
  grossMarginAmount = effectiveRevenue - totalAllCogs
 grossMarginPercent = effectiveRevenue > 0 ? (grossMarginAmount / effectiveRevenue) ×
  mePercent = displayPrice > 0 ? (totalMe / displayPrice) × 100 : 0
```

State Management

- All input values live in `CalculatorContext` so they persist across page navigations and can be shared with the Reverse Calculator.
- The `resetState` function resets to defaults while preserving targetMe and laborPercent from settings.

Additional Features

- **Round Up**: Toggle to round price to next whole dollar
- **Save Calc**: Opens dialog to name and save the calculation as a quote (stored as snapshot in `quotes` table)
- **Download Image**: Exports the results card as a PNG using `html-to-image`

---

4. Repair Calculator

File: `client/src/pages/RepairCalculator.tsx` **Route**: `/repair`

The Repair Calculator prices individual repair jobs using material markup, labor, sales tax, commission, and financing.

Inputs

Input	Description
Parts Cost	Raw cost of parts
Material Markup	Percentage markup on parts+tax (dropdown: 10%–300%)
Labor Rate	Dollar per hour rate
Labor Hours	Number of hours
Burden %	Employer burden on labor/commission
Sales Tax %	Tax applied to parts
Commission %	As % of total selling price (backed in)
Financing Fee %	As % of total selling price

Input	Description
Highest Tech Pay Rate	For COGS calculation
Production Pay toggle	Switches between “$ + Hours” and “% of Job” labor modes
PSI Mode toggle	Preset values for PSI pricing

Calculation Logic (lines 67–134)

When labor mode is “$ + Hours” (default, `laborMode === "dollars"`):

```
salesTax = partsCost × taxRate
subtotal1 = partsCost + salesTax
markupAmount = subtotal1 × markupRate
subtotal2 = subtotal1 + markupAmount

directLaborCost = laborRate × laborHours
laborBurden = directLaborCost × burdenRate
burdenedLaborCost = directLaborCost + laborBurden
subtotal3 = subtotal2 + burdenedLaborCost

denominator = 1 - commissionRate × (1 + burdenRate) - financingRate
totalPriceCalc = denominator > 0 ? subtotal3 / denominator : subtotal3

directCommission = totalPriceCalc × commissionRate
commissionBurden = directCommission × burdenRate
burdenedCommission = directCommission + commissionBurden
financingFee = totalPriceCalc × financingRate

totalBeforeRounding = subtotal3 + burdenedCommission + financingFee
totalPrice = roundUp ? Math.ceil(totalBeforeRounding) : totalBeforeRounding
```

When labor mode is “% of Job” (`laborMode === "percent"`):

```
salesTax = partsCost × taxRate
subtotal1 = partsCost + salesTax
markupAmount = subtotal1 × markupRate
subtotal2 = subtotal1 + markupAmount

directLaborCost = 0 (commission handles everything)
subtotal3 = subtotal2

denominator = 1 - commissionRate × (1 + burdenRate) - financingRate
totalPriceCalc = denominator > 0 ? subtotal3 / denominator : subtotal3

directCommission = totalPriceCalc × commissionRate
commissionBurden = directCommission × burdenRate
burdenedCommission = directCommission + commissionBurden
financingFee = totalPriceCalc × financingRate
```

```
totalBeforeRounding = subtotal3 + burdenedCommission + financingFee
totalPrice = roundUp ? Math.ceil(totalBeforeRounding) : totalBeforeRounding
```

COGS & Gross Margin Calculation (lines 127–134)

```
techLaborCost = laborMode === "percent" ? 0 : highestTechRate × laborHours
burdenedTechLabor = techLaborCost × (1 + burdenRate)

COGS = partsCost + burdenedTechLabor + burdenedCommission
  (Note: Financing is excluded from COGS — it's a pass-through fee)

grossMargin = totalPrice - COGS
grossMarginPercent = totalPrice > 0 ? (grossMargin / totalPrice) × 100 : 0
```

Margin Health Thresholds (Repair Calculator, lines 138–146)

The Repair Calculator uses fixed thresholds (unlike System Calculator which uses `grossMarginMinPercent` from settings): - GM% >= 50 → “Strong” (green) - GM% >= 40 → “Caution” (amber) - GM% < 40 → “Low” (red)

State Sync

The Repair Calculator syncs its state to `CalculatorContext.repairState` via a `useEffect`, so the Reverse Calculator can copy values from it.

---

5. Reverse Calculator

File: `client/src/pages/ReverseCalculator.tsx` **Route**: `/reverse`

The Reverse Calculator works backwards from a known selling price to determine the resulting gross margin percentage. It has two modes: **System** and **Repair**.

Mode Toggle

- Stored in `localStorage` key `pricebookcalc_reverse_mode`
- Persists per device/browser (not saved to database)
- Toggle UI: pill-style switcher at the top of the page

System Mode

Mirrors the System Calculator’s inputs but replaces Target M&E % with a Selling Price input. Outputs GM% instead of computing a price.

Inputs: Selling Price, Materials, Equipment, Labor (hours or %), Subcontractor, Crane, Permit, Discount ($ or %), OEM Rebate, Utility Rebate

Calculation (lines 94–138):

```
tax = (materials + equipment) × salesTaxRate
totalMe = materials + equipment + tax
totalOtherCogs = subcontractor + crane + permit

Discount handling (differs from System Calculator):
 If discountMode is "percent": discountVal = sellingPrice × discountInput / 100
  If discountMode is "dollar": discountVal = discountInput
  price = max(0, sellingPrice - discountVal)

commissionCost = (price × commissionRate) × (1 + commissionBurdenRate)
financingCost = price × financingRate
laborCost = depends on laborMode (same as System Calculator)
estimatedLabor = laborCost + commissionCost + tax + financingCost

totalAllCogs = totalMe + commissionCost + financingCost + laborCost + totalOtherCogs
effectiveRevenue = price + oemRebate + utilityRebate
grossMarginAmount = effectiveRevenue - totalAllCogs
grossMarginPercent = effectiveRevenue > 0 ? (grossMarginAmount / effectiveRevenue) × 1
mePercent = effectiveRevenue > 0 ? (totalMe / effectiveRevenue) × 100 : 0
```

Repair Mode

Mirrors the Repair Calculator's inputs but adds a Selling Price input. Shows GM% based on entered price.

Inputs: Selling Price, Parts Cost, Material Markup, Labor Hours, Burden %, Sales Tax %, Commission %, Financing Fee %, Actual/Highest Tech Pay Rate, Production Pay toggle

Key difference from Repair Calculator: Uses user-entered selling price directly to compute commission/financing amounts, rather than deriving the price from costs. In Repair Calculator, the price is solved algebraically; in Reverse Repair mode, the price is given and used directly.

Calculation (lines 141–187):

```
repairSalesTax = partsCost × taxRate
repairMaterialsCost = (partsCost + salesTax) + (partsCost + salesTax) × markupRate

If laborMode is "percent":
  laborCost = $0
  commission = sellingPrice × commissionRate (+ burden)
  financing = sellingPrice × financingRate

If laborMode is "dollars":
  laborCost = highestTechRate × laborHours (+ burden)
  commission = sellingPrice × commissionRate (+ burden)
  financing = sellingPrice × financingRate
```

```
COGS = partsCost + burdenedTechLabor + burdenedCommission
grossMargin = sellingPrice - COGS
grossMarginPercent = sellingPrice > 0 ? (grossMargin / sellingPrice) × 100 : 0
```

"Copy from" Buttons

- **System mode**: "Copy from System Calculator" — copies materials, equipment, labor, discount, rebates from the System Calculator's current state via `CalculatorContext.state`
- **Repair mode**: "Copy from Repair Calculator" — copies parts, markup, labor, fees from the Repair Calculator's current state via `CalculatorContext.repairState`
- Both buttons are always visible (not conditional on source data existing)

Reset

Each mode has its own reset function that clears all inputs to defaults.

---

6. Rebates (OEM & Utility)

Files: `client/src/pages/Calculator.tsx`, `client/src/pages/ReverseCalculator.tsx`, `client/src/components/ResultsCard.tsx`, `client/src/contexts/CalculatorContext.tsx`

Rebates are dollar amounts that get added back to revenue for gross margin calculation without changing the selling price.

How It Works

- Two fields: OEM Rebate and Utility Rebate
- Present in both System Calculator and Reverse System mode
- Stored in `CalculatorContext.state` as `oemRebate` and `utilityRebate` strings

Impact on GM%

```
effectiveRevenue = displayPrice + oemRebate + utilityRebate
grossMarginAmount = effectiveRevenue - totalAllCogs
grossMarginPercent = (grossMarginAmount / effectiveRevenue) × 100
```

The selling price displayed to the customer does not change. Rebates only improve the GM% calculation because the company receives additional revenue from the manufacturer/utility.

Display

When rebate values are > 0, a "Rebates (Added to Revenue)" section appears in the ResultsCard cost breakdown with teal-colored highlight styling.

---

7. ResultsCard Component

File: `client/src/components/ResultsCard.tsx`

Shared display component used by the System Calculator to show calculation results.

Props

Prop	Type	Description
`price`	number	Calculated selling price
`estimatedLabor`	number	Estimated labor field value
`breakdown`	object	Itemized costs (materials, equipment, tax, commission, etc.)
`metrics`	object	GM%, GM amount, M&E%
`salesTaxRate`	string	For display in breakdown
`discount`	number	Discount amount
`roundUp`	boolean	Whether rounding is enabled
`onRoundUpChange`	function	Toggle callback
`grossMarginMinPercent`	number	Threshold for margin health indicator
`oemRebate`	number	OEM rebate for display
`utilityRebate`	number	Utility rebate for display

Sections

1. **Primary Result Cards**: Price (with margin health bar) + Estimated Labor Field
2. **Metrics Row**: Gross Margin %, M&E %, Direct Labor $, Other COGS $
3. **Cost Breakdown**: Grouped into M&E, Direct Labor, Other COGS, and Rebates
4. **Pie Chart**: Visual distribution (Recharts) with labeled slices

Margin Health Indicator

```
if GM% >= grossMarginMinPercent → "Strong" (green)
if GM% >= grossMarginMinPercent - 2 → "Caution" (amber)
otherwise → "Low" (red)
```

---

8. Calculator Context

File: `client/src/contexts/CalculatorContext.tsx`

React Context that provides shared state between all three calculators.

State Shape

```tsx
interface CalculatorState {
  materials: string;
```

```
  equipment: string;
  laborHours: string;
  laborPercent: string;
  targetMe: string;
  discount: string;
  discountMode: "dollar" | "percent";
  subcontractor: string;
  crane: string;
  permit: string;
  roundUp: boolean;
  oemRebate: string;
  utilityRebate: string;
}

interface RepairCalculatorState {
  partsCost: string;
  materialMarkup: string;
  laborRate: string;
  laborHours: string;
  burdenPercent: string;
  salesTaxPercent: string;
  commissionPercent: string;
  financingFeePercent: string;
  highestTechRate: string;
  laborMode: "dollars" | "percent";
}
```

How It's Used

- **System Calculator** reads/writes `state`
- **Repair Calculator** writes to `repairState` via `useEffect` (syncs local state to context)
- **Reverse Calculator** reads `state` and `repairState` for "Copy from" functionality
- Wrapped at the app root in `App.tsx` via `<CalculatorProvider>`

---

9. Settings & Per-Device Cache

Files: `client/src/pages/Settings.tsx`, `client/src/hooks/use-settings.ts`, `server/routes.ts`, `server/storage.ts`

Settings Page (`/settings`)

Provides a form to configure all calculator defaults: Target M&E %, Sales Tax, Labor mode/rate, Commission, Financing, Burden rates, and Minimum GM%.

Per-Device Cache

How it works: 1. When user saves settings, values are written to both the database AND `localStorage` (key: `pricebookcalc_settings`) 2. On page load, the `use-settings` hook checks `localStorage` first 3. If local settings exist, they override database values 4. If no local settings exist, database settings are used as defaults 5. This means each device/browser can have its own independent settings

API Endpoints

- `GET /api/settings` — Fetch current settings
- `PUT /api/settings` — Update settings

---

10. Quote Saving & History

Files: `client/src/pages/Calculator.tsx` (save dialog), `client/src/pages/History.tsx`, `server/routes.ts`, `server/storage.ts`

Saving a Quote

From the System Calculator, clicking "Save Calc" opens a dialog where the user names the quote. The current calculated results are saved as a snapshot: - Input values (materials, equipment, labor hours, subcontractor, crane, permit) - Output values (price, GM%, estimated labor, M&E%)

History Page (`/history`)

Displays all saved quotes in a list/table with: - Quote title - Calculated price - GM% - Date saved - Delete action

API Endpoints

- `GET /api/quotes` — List all quotes
- `POST /api/quotes` — Create new quote
- `DELETE /api/quotes/:id` — Delete a quote

Admin: Clear History

In the Admin section of the navigation (after unlocking with PIN), there's a "Clear All History" option that deletes all saved quotes.

---

11. ServiceTitan Integration

Files: `server/servicetitan.ts`, `server/routes.ts`

ServiceTitanAPI Class

Located in `server/servicetitan.ts`, this class handles all communication with the ServiceTitan Pricebook v2 API.

Authentication:

```
OAuth2 client_credentials flow
POST https://auth.servicetitan.io/connect/token
Tokens cached in-memory for 15 minutes per tenant
```

Key Methods: | Method | Purpose | |———-|———| | `getAccessToken()` | Obtain/refresh OAuth token | | `getCategories()` | Fetch pricebook category tree | | `getServices(categoryId)` | Fetch services by category | | `getMaterials(onProgress)` | Fetch all materials (paginated) | | `getEquipment(onProgress)` | Fetch all equipment (paginated) | | `updateService(id, data)` | Update service price/hours | | `createService(data)` | Create new service | | `uploadServiceImage(id, file)` | Upload image to service | | `getGLAccounts()` | Fetch GL account list |

API Field Mapping (ServiceTitan ↔ PricebookCalc)

ServiceTitan Field	API Property	Updatable?
Static Price	`price`	Yes
Estimated Labor Cost	`cost`	No (calculated by ST from Hours × internal rate)
Hours	`hours`	Yes

Tenant Management

Page: `client/src/pages/Tenants.tsx` (Route: `/tenants`, protected) - CRUD operations for ServiceTitan tenant credentials - Credentials are masked in API responses (never sent to frontend in full) - Each tenant has: name, tenantId, clientId, clientSecret, appKey - Tenants can be assigned a location (for sales tax) - Sort order controls display ordering

API Endpoints

- `GET /api/tenants` — List all tenants (credentials masked)
- `POST /api/tenants` — Create tenant
- `PUT /api/tenants/:id` — Update tenant
- `DELETE /api/tenants/:id` — Delete tenant

---

12. Push to ServiceTitan (Update Pricebook)

Files: `client/src/pages/PushToServiceTitan.tsx`, `server/pushProcessor.ts`, `server/routes.ts` **Route**: `/push` (password-protected)

The "Update Pricebook" tab on the Push to ServiceTitan page lets users push calculated M&E pricing to existing services in ServiceTitan. The page also includes a collapsible **Tax Regions** management section (between tenant selection and the tabs) for inline add/edit/delete of tax regions without navigating away.

How It Works

1. **User selects**: Tenant, categories, and confirms settings
2. **Backend receives push request** and creates a `pushHistory` record with status "pending"
3. **Backend responds immediately** (fire-and-forget) — processing happens in background
4. **Push processor** (`server/pushProcessor.ts`) runs:
   - Fetches services from selected categories
   - Fetches all materials and equipment for cost lookup
   - For each service with M&E > $0:
     - Calculates `newPrice = totalMe / (targetMePercent / 100)`
     - Calculates `estimatedLabor = laborCost + commissionCost + tax + financingCost`
     - Updates service `price` via API (cost is read-only in ServiceTitan)
   - Stores original values for potential revert

Price Calculation per Service

```
materialsCost = sum of (material.cost × material.quantity) for linked materials
equipmentCost = sum of (equipment.cost × equipment.quantity) for linked equipment
rawMeCost = materialsCost + equipmentCost
tax = rawMeCost × salesTaxRate
totalMe = rawMeCost + tax
newPrice = totalMe / (targetMePercent / 100)
```

Category Filtering

The push processor intelligently handles parent-child category relationships: - Fetches the full category tree to understand hierarchy - If a parent category AND its child are both selected, only the child is queried (parent query would include children redundantly) - Services are filtered to only include those whose category exactly matches a selected category

Progress Tracking

Uses Server-Sent Events (SSE) via `server/pushProgress.ts`: - Client opens EventSource connection to `/api/servicetitan/:tenantId/push-progress` - Server

emits progress events for phases: services, materials, equipment, updating, complete - Frontend displays real-time progress bars and status messages

Revert

Push history records store `originalValues` (JSON array of original prices). The revert endpoint restores these values.

---

13. Push Queue & Rate Limiting

File: `server/pushQueue.ts`

Concurrency Control

- Maximum 3 concurrent pushes (`MAX_CONCURRENT_PUSHES = 3`)
- Each push processes up to 20 service updates in parallel (`CONCURRENCY = 20` in pushProcessor)
- Total maximum: 20 × 3 = 60 requests/second (within ServiceTitan's 60 req/sec limit)

Queue Behavior

- If 3 pushes are already active, new pushes are queued
- Queue processes in FIFO order
- When an active push completes, the next queued push starts automatically
- `pushQueue` extends `EventEmitter` — emits 'ready' events to trigger dequeued pushes

Exponential Backoff Retry

The `withRetry` function handles ServiceTitan 429 (rate limit) errors:

```
For each retry attempt (max 3):
  delay = baseDelay × 2^attempt
  (1000ms, 2000ms, 4000ms)
Only retries on 429/rate-limit errors; other errors thrown immediately
```

Queue Status API

- `GET /api/servicetitan/push-queue/status` — Returns active and queued push info

---

14. New System Task & New Repair Task

File: `client/src/pages/PushToServiceTitan.tsx` (tabs 2 and 3)

New System Task Tab

Creates new system installation services in ServiceTitan with M&E-based pricing.

Includes: - Price calculator (Materials/Equipment cost → price via M&E target) - Form fields: Code, Name, Description, Warranty Description - Checkboxes: Taxable, Allow Discount Codes, Allow Membership Discounts - Hours field, GL Account dropdown (searchable), Category selection - Image upload support - PSI Mode toggle

New Repair Task Tab

Creates new repair services in ServiceTitan with labor/commission pricing.

Includes: - Repair price calculator (Parts, Markup, Labor, Tax, Commission, Financing) - Same form fields as System Task - Production Pay toggle ($ + Hours vs % of Job) - PSI Mode toggle - Real-time price breakdown with gross margin indicator

API Endpoints

- `GET /api/servicetitan/:tenantId/gl-accounts` — Fetch GL accounts for dropdown
- `POST /api/servicetitan/:tenantId/services` — Create new service
- `POST /api/servicetitan/:tenantId/services/:serviceId/image` — Upload image to service

---

15. PSI Mode Toggle

File: `client/src/pages/PushToServiceTitan.tsx`, `client/src/pages/RepairCalculator.tsx`

PSI Mode is a quick-preset toggle that applies a specific set of values for PSI (Performance Service Institute) pricing standards.

New System Task PSI Mode

- Switches GL Account from “Revenue” (default) to “Sales”

New Repair Task / Repair Calculator PSI Mode

When PSI ON: | Setting | Value | |———|——-| | Material Markup | 40% | | Labor Rate | $600 | | Sales Tax | 7.4% | | GL Account | “Sales” |

When PSI OFF (defaults): | Setting | Value | |———|——-| | Material Markup | 200% | | Labor Rate | $175 (from settings) | | Sales Tax | From tenant’s assigned location (fallback 6.35%) | | GL Account | “Revenue” | | Financing Fee | 4% |

---

16. Location-Based Sales Tax (Tax Regions)

Files: `client/src/pages/Locations.tsx`, `client/src/pages/PushToServiceTitan.tsx`, `server/routes.ts`, `server/storage.ts`, `shared/schema.ts` **Route**: `/locations` (password-protected admin page), also accessible inline on `/push`

Purpose

A reference library that maps location names (tax regions) to their local sales tax percentages. Tenants can be assigned a location, and when that tenant is selected on the Push to ServiceTitan page, the tax rate auto-populates into **all three tabs** (Update Pricebook, New System Task, New Repair Task).

Managing Tax Regions

Tax regions can be managed in two places: 1. **Admin Locations page** (`/locations`) — full-page CRUD management 2. **Push to ServiceTitan page** (`/push`) — collapsible inline "Tax Regions" section between tenant selection and the main tabs, allowing add/edit/delete without navigating away

Both interfaces share the same `/api/locations` endpoints and data.

Seeded Locations

Location	Sales Tax
Gaithersburg MD	6.00%
Hartford CT	6.35%
Gypsum CO	7.40%
Jupiter FL	7.00%
Denver CO	9.15%
Houston TX	8.25%

How Tax Auto-Populates

1. User selects a tenant on the Push to ServiceTitan page
2. If that tenant has an assigned `locationId`, the location's `salesTaxPercent` is fetched
3. The tax rate is set on **both** `salesTaxPercent` (used by Update and New System tabs) and `repairSalesTaxPercent` (used by New Repair tab)
4. If PSI mode is toggled off, the location-based tax is restored
5. Fallback: global settings value, or 6.35% if no location is assigned

API Endpoints

- `GET /api/locations` — List all locations
- `POST /api/locations` — Create location
- `PUT /api/locations/:id` — Update location

- `DELETE /api/locations/:id` — Delete location (also clears `locationId` on any associated tenants)

---

17. Commission & Labor Modes

Commission

Commission is **always** calculated as "% of Job" — it's a percentage of the total selling price that gets backed into the total. There is no toggle for commission mode; it's always this way.

```
directCommission = totalPrice × commissionRate
commissionBurden = directCommission × burdenRate
burdenedCommission = directCommission + commissionBurden
```

Labor Mode (Production Pay Toggle)

Files: `client/src/pages/RepairCalculator.tsx`, `client/src/pages/ReverseCalculator.tsx`, `client/src/pages/PushToServiceTitan.tsx`

A Switch component labeled "Production Pay" toggles between two modes:

"$ + Hours" Mode (default)

- Labor Cost = Labor Rate × Labor Hours
- Commission is a separate % of job
- Both labor and commission have burden applied independently
- Formula: `T = subtotal3 / (1 - commissionRate×(1+burdenRate) - financingRate)`

"% of Job" Mode

- Labor cost is $0 — commission handles everything (labor + commission combined)
- No separate labor inputs shown
- Commission % field covers both labor and commission
- Formula: `T = subtotal2 / (1 - commissionRate×(1+burdenRate) - financingRate)`
- Price breakdown hides Labor line and relabels Commission as "Commission incl. Labor"
- Highest Tech Pay Rate and COGS tech labor are zeroed (no hours to calculate)

---

18. Authentication & Access Control

ServiceTitan Section (Password)

Files: `client/src/contexts/ServiceTitanAuthContext.tsx`, `client/src/components/Passwor`
`server/routes.ts`

Protected pages: Push to ServiceTitan, Push History, Tenants, Locations

How it works: 1. Pages are wrapped in `<PasswordGate>` component in `App.tsx` 2. `PasswordGate` checks `useServiceTitanAuth()` context 3. If not unlocked, shows a password prompt card 4. Password is verified server-side against `settings.serviceTitanPassword` 5. On success, the client's IP is stored in a server-side `Map` with 8-hour expiration 6. Subsequent requests from the same IP are auto-authenticated

Server-side IP tracking:

```
const unlockedIPs = new Map<string, number>(); // IP -> expiration timestamp
const AUTH_DURATION = 8 * 60 * 60 * 1000; // 8 hours
```

API Endpoints: - `GET /api/admin/check-auth` — Check if current IP is authenticated - `POST /api/admin/verify-password` — Verify password and unlock IP - `POST /api/admin/change-password` — Change the ServiceTitan password

Admin Section (PIN)

Files: `client/src/contexts/AdminAuthContext.tsx`, `client/src/components/Navigation.tsx`

Protected features: Change Password, Change PIN, Clear History, Lock Push Section

How it works: 1. Admin section is in the sidebar navigation 2. Expanding it shows a PIN input form 3. PIN is verified server-side against `settings.adminPin` 4. Same IP-based authentication with 8-hour expiration 5. Once unlocked, admin actions are available inline in the sidebar

Admin capabilities: - Change ServiceTitan password - Change Admin PIN - Clear all saved quote history - Lock/unlock the ServiceTitan push section

API Endpoints: - `GET /api/admin/check-admin-auth` — Check admin auth status - `POST /api/admin/verify-pin` — Verify PIN - `POST /api/admin/change-pin` — Change admin PIN - `DELETE /api/admin/clear-history` — Delete all quotes

---

19. Version Check Banner

File: `client/src/components/VersionCheck.tsx`, `server/routes.ts`

Notifies users when a new version of the app has been published.

How It Works

1. Server generates a unique build ID (`Date.now().toString()`) at startup

2. `/api/version` endpoint returns this build ID
3. Frontend component polls `/api/version` every 30 seconds
4. On first load, it stores the build ID in a `useRef`
5. If a subsequent poll returns a different build ID, a sticky banner appears
6. Banner says "A new version of PricebookCalc is available" with a "Refresh Now" button
7. Button calls `window.location.reload()` to load the new version

Styling

- Sticky banner at the very top of the page (`z-index: 9999`)
- Primary background color with spinning refresh icon
- Appears on all pages (component is rendered at the app root level)

---

20. Navigation & Routing

File: `client/src/components/Navigation.tsx`, `client/src/App.tsx`

Route Map

Path	Page	Protection
`/`	System Calculator	None
`/repair`	Repair Calculator	None
`/reverse`	Reverse Calculator	None
`/push`	Push to ServiceTitan	Password
`/push-history`	Push History	Password
`/tenants`	Manage Tenants	Password
`/locations`	Manage Locations	Password
`/history`	Saved Quotes	None
`/settings`	Calculator Settings	None

Navigation Component

- **Desktop**: Fixed sidebar on the left (264px wide)
- **Mobile**: Hamburger menu that slides in from the left
- **ServiceTitan section**: Collapsible group with lock icon when locked
- **Admin section**: Collapsible group at bottom with PIN-gated access

Provider Hierarchy (App.tsx)

```
QueryClientProvider
  └── TooltipProvider
      └── ServiceTitanAuthProvider
          └── AdminAuthProvider
```

```
└── CalculatorProvider
    ├── VersionCheck
    ├── Toaster
    └── Router
        ├── Navigation (sidebar)
        └── <main> (page content)
```

Appendix: Common Patterns

InputMoney Component

File: `client/src/components/InputMoney.tsx`

Reusable input with dollar sign prefix, consistent styling, and label. Used across all calculators for currency/numeric inputs.

TanStack Query Usage

- Queries use default `queryFn` (no need to define fetch logic)
- Mutations use `apiRequest` from `@/lib/queryClient`
- Cache invalidation by `queryKey` after mutations
- Hierarchical keys: `['/api/resource', id]` for proper invalidation

Drizzle ORM Pattern

- Schema defined in `shared/schema.ts`
- Insert schemas created with `createInsertSchema` from `drizzle-zod`
- Storage operations in `server/storage.ts` implement `IStorage` interface
- All CRUD routed through storage interface (routes stay thin)